

PRD — Product Requirements Document

AI-Powered Smart Complaint & Support Management System

1. Product Overview

Product Name

SupportSphere AI

Tagline

“Smart complaint management powered by AI.”

Product Summary

SupportSphere AI is a full-stack web application that helps organizations manage complaints, support requests, and issue tracking efficiently using AI-powered automation.

The platform allows users to:

- Submit complaints/support tickets
- Track issue status
- Receive AI-generated assistance
- Upload evidence/screenshots
- Download PDF reports
- Communicate with administrators

Administrators can:

- Manage complaints
- Assign priorities
- Monitor analytics
- Generate reports
- Improve resolution efficiency

2. Problem Statement

Many colleges, startups, hostels, coaching institutes, and small businesses still manage complaints manually through:

- WhatsApp
- Emails
- Phone calls
- Paper records
- Spreadsheets

This creates problems such as:

- Lost complaints
- Delayed responses
- No tracking system
- Poor communication
- No analytics/reporting
- No prioritization
- No accountability

There is a need for a centralized, intelligent complaint management platform.

3. Vision Statement

To create a modern AI-powered support system that simplifies complaint handling, improves response times, and provides actionable insights through automation and analytics.

4. Goals & Objectives

Primary Goals

- Centralize complaint management
- Improve issue resolution efficiency
- Reduce manual workload
- Provide transparent tracking
- Introduce AI-powered automation

Business Goals

- Improve user satisfaction
- Reduce response time
- Enable data-driven decision making
- Create scalable support infrastructure

5. Target Users

Primary Users

User Type	Description
Students	Raise hostel, lab, internet, or academic complaints
Employees	Report technical or operational issues
Customers	Submit support tickets
Admins	Manage and resolve complaints

Secondary Users

User Type	Description
Support Staff	Handle assigned complaints
Managers	View analytics and reports

6. User Personas

Persona 1 — Student User

Name: Rahul

Age: 20

Problem: Internet not working in hostel

Goal: Quickly report issue and track resolution status

Persona 2 — Admin

Name: Priya

Role: Support Manager

Goal: Manage multiple complaints efficiently and reduce pending tickets

7. Core Features

A. Authentication System

Features

- User registration
- Login/logout
- JWT authentication
- Password encryption
- Role-based access control

Roles

- User
- Admin
- Support Staff

B. Complaint/Ticket Management

User Features

- Create complaint
- Add title & description
- Select category
- Upload screenshots/images
- View complaint history
- Track status

Ticket Statuses

- Pending
- In Progress
- Resolved
- Closed

C. AI-Powered Assistance

AI Features

- Automatic category detection
- Priority prediction
- AI-generated response suggestions
- Smart complaint summarization

Example

Input:

“Lab internet is not working.”

AI Output:

- Category → Technical
- Priority → High
- Suggested Action → Check router/network switch

D. Admin Dashboard

Features

- View all tickets
- Search/filter complaints
- Assign priorities
- Update statuses
- Respond to users
- View analytics

E. Analytics Dashboard

Metrics

- Total complaints
- Resolved vs Pending
- Category distribution
- Monthly reports
- Average resolution time

F. File Upload System

Features

- Upload screenshots
- Upload supporting documents
- Cloud storage integration

G. PDF Report Generation

Reports

- Complaint summary
- Monthly analytics
- User complaint history

8. Nice-to-Have Features

Feature	Priority
Dark Mode	Medium
Real-time Notifications	Medium
Email Alerts	Medium
Chat Support	Low
Multi-language Support	Low
Mobile App	Future

9. User Stories

Authentication

User Story 1

As a user, I want to create an account so that I can submit complaints securely.

Complaint Creation

User Story 2

As a user, I want to submit a complaint with images so that support staff can understand the issue better.

Tracking

User Story 3

As a user, I want to track complaint progress so that I know when my issue will be resolved.

Admin Management

User Story 4

As an admin, I want to update ticket statuses so that users receive accurate progress updates.

AI Assistance

User Story 5

As an admin, I want AI-generated suggestions so that complaints can be resolved faster.

10. Functional Requirements

User Requirements

Requirement	Priority
User registration/login	High
Create complaint	High
Upload attachments	High
Track status	High

View complaint history Medium

Admin Requirements

Requirement	Priority
Manage complaints	High
Update statuses	High
View analytics	High
Generate reports	Medium

AI Requirements

Requirement	Priority
Auto categorization	Medium
AI suggestions	Medium
Priority prediction	Medium

11. Non-Functional Requirements

Requirement	Description
Performance	Pages load under 3 seconds
Security	JWT + encrypted passwords

Scalability	Support 10,000+ complaints
Availability	99% uptime
Responsiveness	Mobile + desktop compatible

12. Technical Constraints

Constraint	Description
Budget	Use free-tier services
Hosting	Cloud deployment required
Database	NoSQL preferred
Timeline	Complete MVP within 7 days

13. Success Metrics

Metric	Goal
Complaint submission success rate	95%
Average resolution time	Reduced by 40%
User satisfaction	Above 80%
Monthly active users	1000+
System uptime	99%

14. Risks & Challenges

Risk	Solution
AI inaccuracies	Manual admin review
Large file uploads	File size limits
Unauthorized access	JWT authentication
High server load	Pagination & caching

15. Future Scope

Future versions may include:

- Mobile application
- AI chatbot
- Voice complaint submission
- Real-time messaging
- Predictive analytics
- Multi-organization support

16. MVP Scope (7-Day Build)

Included in MVP

- ✓ Authentication
- ✓ Complaint system
- ✓ Admin dashboard
- ✓ AI categorization
- ✓ Analytics
- ✓ PDF reports
- ✓ Deployment

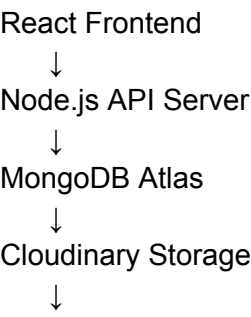
Excluded from MVP

- × Mobile app
- × Real-time chat
- × Advanced AI analytics
- × Multi-language support

17. Proposed Tech Stack

Layer	Technology
Frontend	React.js + Tailwind CSS
Backend	Node.js + Express
Database	MongoDB Atlas
Authentication	JWT
File Storage	Cloudinary
Charts	Chart.js
AI	Gemini API
Deployment	Vercel + Render

18. Deployment Architecture



19. Development Timeline

Day	Tasks
Day 1	Setup + Authentication
Day 2	Complaint CRUD
Day 3	Admin Dashboard
Day 4	AI Integration
Day 5	Analytics
Day 6	PDF + Uploads
Day 7	Testing + Deployment

20. Final Summary

SupportSphere AI is a scalable AI-powered complaint and support management system designed to modernize issue tracking for organizations.

The platform combines:

- MERN Stack
- AI automation
- Analytics
- PDF reporting
- Secure authentication
- Cloud deployment

to provide a professional real-world solution suitable for:

- Colleges
- Startups
- Businesses

- Hostels
- Coaching institutes
- Technical support teams

TRD — Technical Requirements Document

SupportSphere AI — AI-Powered Smart Complaint & Support Management System

1. Document Overview

Purpose

This Technical Requirements Document (TRD) defines the complete technical architecture, development standards, infrastructure, integrations, security requirements, and implementation details for the SupportSphere AI platform.

This document serves as the technical blueprint for:

- Developers
- AI coding agents
- Designers
- DevOps engineers
- Project collaborators

2. System Overview

System Name

SupportSphere AI

System Type

AI-Powered Complaint & Support Management Platform

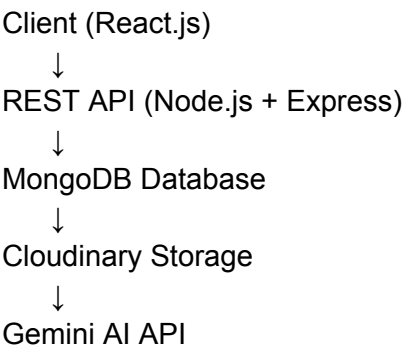
Architecture Type

Full Stack Web Application (MERN Architecture)

Primary Objectives

- Complaint tracking
- AI-assisted issue handling
- Admin analytics
- Secure authentication
- PDF report generation
- Cloud deployment

3. High-Level Architecture



4. Technology Stack

Layer	Technology
Frontend	React.js
Styling	Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB Atlas

Authentication	JWT + bcrypt
State Management	Context API / Redux Toolkit
Charts	Chart.js
File Upload	Multer
Cloud Storage	Cloudinary
AI Integration	Gemini API
PDF Generation	jsPDF / PDFKit
Deployment Frontend	Vercel
Deployment Backend	Render
API Testing	Postman
Version Control	Git + GitHub

5. Frontend Requirements

Frontend Framework

Technology

- React.js (Vite preferred)

UI Framework

Styling

- Tailwind CSS

Frontend Features

Feature	Requirement
Responsive Design	Mobile + Desktop
Dark Mode	Optional
Protected Routes	Required
Form Validation	Required
Toast Notifications	Required
Loading States	Required

Frontend Pages

Page	Purpose
Landing Page	Product overview
Login	Authentication
Register	User signup
Dashboard	Main user panel
Complaint Details	Complaint tracking
Admin Dashboard	Complaint management

Analytics

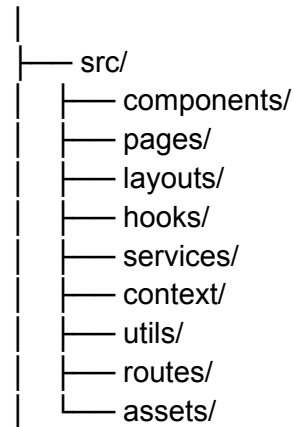
Reports & charts

Settings

User preferences

Frontend Folder Structure

client/



6. Backend Requirements

Backend Framework

Technology

- Node.js
- Express.js

API Architecture

Type

RESTful API

Backend Features

Feature	Requirement
JWT Authentication	Required
Role-Based Access	Required
File Upload API	Required
AI Processing	Required
PDF Generation	Required
Validation Middleware	Required
Error Handling	Required

Backend Folder Structure

```
server/  
|  
├── controllers/  
├── routes/  
├── models/  
├── middleware/  
├── services/  
├── utils/  
├── config/  
├── validators/  
└── uploads/
```

7. Database Requirements

Database Technology

Database

MongoDB Atlas

Database Design Goals

- Scalability
- Fast querying
- Flexible schema
- Cloud accessibility

Collections

Collection	Purpose
users	Stores user data
tickets	Stores complaints
comments	Stores admin/user replies
analytics	Stores generated analytics
notifications	Stores alerts

User Schema

```
{
  _id,
  name,
  email,
  password,
  role,
  profileImage,
  createdAt,
```

```
    updatedAt
  }
}
```

Ticket Schema

```
{
  _id,
  title,
  description,
  category,
  priority,
  status,
  attachments,
  aiSuggestion,
  createdBy,
  assignedTo,
  createdAt,
  updatedAt
}
```

Comment Schema

```
{
  _id,
  ticketId,
  userId,
  message,
  createdAt
}
```

8. Authentication & Authorization

Authentication Method

Technology

- JWT Authentication

Password Security

Encryption

- bcrypt hashing

Role-Based Access Control (RBAC)

Role	Permissions
User	Create/view complaints
Admin	Manage all complaints
Support Staff	Update assigned complaints

Protected Routes

Route	Access
/dashboard	Authenticated users
/admin	Admin only
/tickets/create	Logged-in users

9. API Requirements

Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
GET	/api/auth/profile	Get profile

Ticket APIs

Method	Endpoint	Purpose
POST	/api/tickets	Create ticket
GET	/api/tickets	Get tickets
GET	/api/tickets/:id	Get single ticket
PUT	/api/tickets/:id	Update ticket
DELETE	/api/tickets/:id	Delete ticket

Admin APIs

Method	Endpoint	Purpose
GET	/api/admin/analytics	Analytics
PUT	/api/admin/status	Update status
GET	/api/admin/reports	Generate reports

AI APIs

Method	Endpoint	Purpose
POST	/api/ai/analyze	AI categorization
POST	/api/ai/suggest	AI suggestions

10. AI Integration Requirements

AI Provider

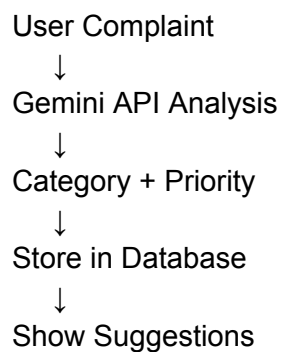
Primary

- Gemini API

AI Functionalities

Function	Description
Categorization	Detect complaint type
Priority Prediction	High/medium/low
Summarization	Short complaint summary
Suggested Responses	AI-generated support reply

AI Processing Flow



11. File Upload System

Upload Features

Feature	Requirement
Image Upload	Required
PDF Upload	Optional
File Validation	Required
Max File Size	5MB

Storage Provider

Clouldinary

Accepted Formats

Type	Formats
Images	JPG, PNG, WEBP
Documents	PDF

12. Analytics System

Analytics Requirements

Metric	Description
Total Tickets	Count

Pending Tickets	Count
Resolved Tickets	Count
Category Distribution	Pie chart
Monthly Trends	Line graph

Visualization Tools

Technology

- Chart.js

13. PDF Generation Requirements

PDF Features

Report	Description
Ticket Report	Complaint summary
Analytics Report	Admin statistics
User History	Complaint history

PDF Libraries

Library	Purpose
---------	---------

jsPDF Frontend PDFs

PDFKit Backend reports

14. Security Requirements

Security Features

Security Feature	Requirement
Password Hashing	Required
JWT Authentication	Required
Input Validation	Required
Rate Limiting	Required
XSS Protection	Required
CORS Configuration	Required
Environment Variables	Required

Security Middleware

Middleware	Purpose
helmet	Security headers
express-rate-limit	API protection

cors	Cross-origin security
dotenv	Environment variables

15. Performance Requirements

Requirement	Target
API Response Time	< 500ms
Initial Page Load	< 3 seconds
Concurrent Users	1000+
Uptime	99%

16. Error Handling Requirements

Frontend Error Handling

Error Type	Action
API Error	Toast notification
Validation Error	Inline message
Authentication Error	Redirect login

Backend Error Handling

Error	Response
Unauthorized	401
Forbidden	403
Validation Failure	400
Server Failure	500

17. Deployment Requirements

Frontend Hosting

Platform

- [Vercel](#)

Backend Hosting

Platform

- [Render](#)

Database Hosting

Platform

- [MongoDB Atlas](#)

Deployment Flow

GitHub Repository
↓
Frontend → Vercel
Backend → Render
Database → MongoDB Atlas

18. Environment Variables

Frontend .env

```
VITE_API_URL=  
VITE_GEMINI_API_KEY=
```

Backend .env

```
PORT=  
MONGO_URI=  
JWT_SECRET=  
GEMINI_API_KEY=  
CLOUDINARY_NAME=  
CLOUDINARY_KEY=  
CLOUDINARY_SECRET=
```

19. Development Standards

Coding Standards

Standard	Requirement
ESLint	Required
Prettier	Required

Modular Code	Required
--------------	----------

Reusable Components	Required
---------------------	----------

Git Workflow

```
main
├── frontend
├── backend
├── feature/auth
├── feature/dashboard
└── feature/analytics
```

20. Testing Requirements

Frontend Testing

Test	Tool
UI Testing	React Testing Library
Form Validation	Jest

Backend Testing

Test	Tool
API Testing	Postman
Unit Testing	Jest

21. Scalability Considerations

Future scalability support:

- Microservices migration
- Redis caching
- Docker support
- Kubernetes deployment
- WebSocket real-time notifications

22. Browser Compatibility

Browser	Support
Chrome	Yes
Edge	Yes
Firefox	Yes
Safari	Yes

23. Accessibility Requirements

Requirement	Description
Keyboard Navigation	Required
Semantic HTML	Required
Responsive Layout	Required
Color Contrast	WCAG compliant

24. MVP Scope

Included

- ✓ Authentication
- ✓ Ticket management
- ✓ AI categorization
- ✓ Admin dashboard
- ✓ Analytics
- ✓ PDF reports
- ✓ Deployment

Excluded

- × Mobile app
- × Real-time chat
- × Voice assistant
- × Multi-language support

25. Final Technical Summary

SupportSphere AI will be developed as a scalable MERN-stack application using:

- React frontend
- Node.js backend
- MongoDB Atlas
- AI integrations
- Cloud deployment
- Secure authentication
- Analytics & reporting

The architecture is designed for:

- Rapid development
- AI-assisted coding
- Scalability
- Maintainability
- Production deployment

App Flow — User Navigation

SupportSphere AI — AI-Powered Smart Complaint & Support Management System

1. Overview

The App Flow document defines:

- User navigation
- Screen hierarchy
- Authentication flow
- Complaint management workflow
- Admin workflow
- Error handling flow
- Protected routes
- User interaction lifecycle

This ensures:

- Smooth navigation
- Logical user experience
- Consistent routing
- Clear backend integration
- Proper access control

2. Application Navigation Overview

Landing Page



Login / Register



Authentication



Dashboard



Complaint Management



Analytics / Reports

↓
Settings / Profile

3. User Roles

Role	Access
Guest	View landing page
User	Submit & track complaints
Admin	Manage all complaints
Support Staff	Resolve assigned complaints

4. Global Navigation Structure

Navbar

- Home
- Features
- About
- Login
- Register

Authenticated Navbar

- Dashboard
- My Complaints
- Analytics
- Notifications
- Settings
- Logout

5. Guest User Flow

Purpose

Allows non-authenticated users to:

- Explore the platform
- Learn features
- Register/Login

Guest Navigation Flow

Landing Page



About Platform



Features Section



Login/Register

Guest Screens

Screen	Purpose
Landing Page	Product introduction
About	Platform details
Features	Showcase capabilities
Login	Existing user access
Register	New account creation

6. Authentication Flow

Authentication Process

Register/Login
↓
Input Validation
↓
JWT Authentication
↓
Role Detection
↓
Dashboard Redirect

Registration Flow

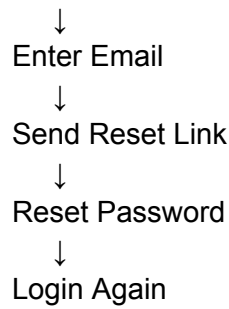
Register Page
↓
Enter User Details
↓
Validate Form
↓
Create Account
↓
Store JWT Token
↓
Redirect Dashboard

Login Flow

Login Page
↓
Enter Credentials
↓
Validate Credentials
↓
Generate JWT Token
↓
Redirect Dashboard

Forgot Password Flow (Optional)

Forgot Password



7. User Dashboard Flow

Dashboard Purpose

The dashboard acts as:

- User control center
- Complaint overview
- Analytics display
- Quick action panel

User Dashboard Navigation



Dashboard Components

Component	Purpose
Complaint Stats	Total/pending/resolved

Recent Complaints	Quick overview
AI Suggestions	Smart assistance
Notifications	Updates
Quick Actions	Fast navigation

8. Complaint Creation Flow

Complaint Submission Process

Create Complaint



Enter Title



Enter Description



Select Category



Upload Attachments



Submit Complaint



AI Analysis



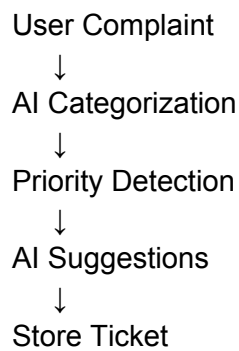
Ticket Created

Complaint Categories

Category	Example
Technical	Internet/Lab issue
Hostel	Maintenance

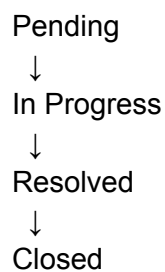
Academic	Faculty issue
Administrative	Documentation
Security	Safety concerns

AI Processing Flow

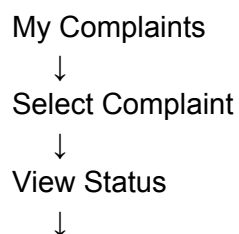


9. Complaint Tracking Flow

Complaint Lifecycle



Complaint Details Navigation



View AI Suggestions



View Admin Replies

Complaint Detail Components

Component	Purpose
Ticket Info	Complaint details
Status Tracker	Progress tracking
Attachments	Uploaded files
Replies	Admin responses
Timeline	Activity history

10. Admin Flow

Admin Dashboard Purpose

Allows admins to:

- Manage complaints
- Assign priorities
- Monitor analytics
- Generate reports

Admin Navigation Structure

Admin Dashboard

- |
- | — All Complaints
- | — Complaint Details

- Analytics
- User Management
- Reports
- Settings

Admin Workflow

View Complaints



Filter/Search



Open Ticket



Update Status



Assign Priority



Reply to User



Resolve Ticket

Admin Filters

Filter	Purpose
Status	Pending/Resolved
Category	Technical/Hostel
Priority	High/Medium/Low
Date	Time-based filtering

11. Support Staff Flow

Support Staff Responsibilities

- View assigned complaints
- Respond to users
- Update progress
- Resolve issues

Support Staff Navigation

Assigned Complaints



Open Ticket



Add Response



Update Status



Resolve Issue

12. Analytics Flow

Analytics Navigation

Dashboard



Analytics Page



View Charts



Generate Reports



Download PDF

Analytics Components

Component	Purpose
Pie Charts	Category distribution

Line Charts Monthly trends

KPI Cards Key statistics

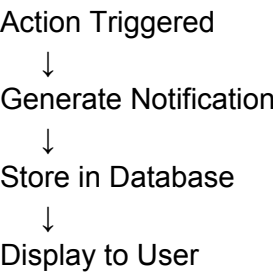
Reports Exportable
summaries

13. Notification Flow

Notification Types

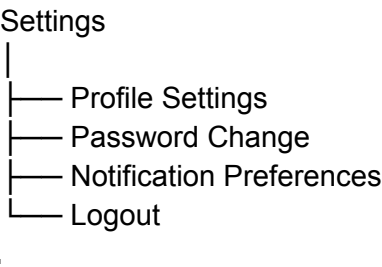
Notification	Trigger
Ticket Created	Complaint submission
Status Updated	Admin update
Ticket Resolved	Resolution completed
New Reply	Admin response

Notification Flow



14. Settings Flow

Settings Navigation

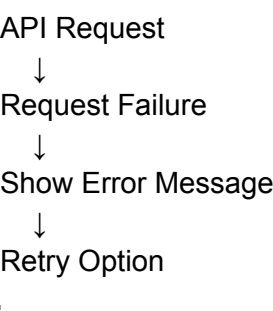


Settings Features

Feature	Purpose
Update Profile	Modify user data
Change Password	Security
Notification Settings	Preferences
Theme Settings	Dark/light mode

15. Error Handling Flow

Frontend Error Flow



Authentication Error Flow

Invalid Token

↓
Logout User
↓
Redirect Login

Common Error Pages

Error	Screen
404	Page not found
401	Unauthorized
500	Server error

16. Protected Route Flow

Route Security

User Access Request
↓
JWT Validation
↓
Role Verification
↓
Grant / Deny Access

Route Permissions

Route	Access
/dashboard	User/Admin
/admin	Admin only

/analytics Admin only

/tickets Authenticated
 users

17. Mobile Responsive Flow

Mobile Navigation

Mobile Navbar



Hamburger Menu



Dashboard Navigation

Mobile Requirements

Requirement	Description
Responsive Layout	Required
Touch Friendly	Required
Adaptive Charts	Required
Mobile Forms	Optimized

18. Logout Flow

Click Logout



Clear JWT Token



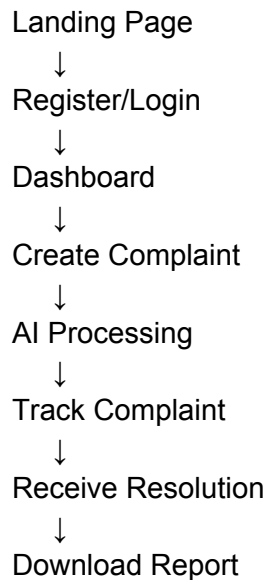
Clear Session



Redirect Landing Page

19. Complete User Journey

End-to-End Flow



20. Future Navigation Enhancements

Future improvements:

- Real-time chat
- AI chatbot assistant
- Multi-language navigation
- Voice complaint flow
- Mobile application flow
- Push notifications

21. Final Navigation Summary

SupportSphere AI navigation is designed for:

- Simplicity
- Scalability

- Clear workflows
- Fast complaint handling
- Role-based access
- Smooth user experience

The flow supports:

- Users
- Admins
- Support staff
- AI-assisted workflows
- Analytics & reporting

through a secure and responsive navigation structure.

UI/UX Design Brief

SupportSphere AI — AI-Powered Smart Complaint & Support Management System

1. Document Overview

Purpose

This UI/UX Design Brief defines the complete visual and user experience guidelines for SupportSphere AI.

The document ensures:

- Consistent interface design
- Professional appearance
- Smooth user experience
- Accessibility compliance
- Responsive layouts
- Modern SaaS-style UI

This brief will guide:

- Frontend developers
- Designers
- AI coding agents
- Component architecture
- Design systems

2. Product Design Vision

Design Philosophy

SupportSphere AI should feel:

- Modern
- Professional

- Clean
- Fast
- Minimal
- Intelligent
- Enterprise-ready

The design should resemble modern SaaS platforms like:

- Linear
- Notion
- Slack
- Jira

3. Design Goals

Goal	Description
Simplicity	Easy navigation and clean layouts
Efficiency	Fast complaint management
Accessibility	Easy for all users
Responsiveness	Mobile + desktop support
Consistency	Unified components
Scalability	Easy future expansion

4. Overall Design Style

Element	Style
Theme	Modern SaaS
Layout	Minimal & spacious

Feel	Professional
UI Density	Medium
Visual Hierarchy	Strong
Animation	Smooth & subtle
Corner Radius	Rounded
Shadows	Soft

5. Color Palette

Primary Colors

Color Role	HEX
Primary Blue	#2563EB
Secondary Purple	#7C3AED
Accent Cyan	#06B6D4

Neutral Colors

Color Role	HEX
Background Dark	#0F172A
Surface Dark	#1E293B

Border	#334155
Text Primary	#F8FAFC
Text Secondary	#CBD5E1

Status Colors

Status	Color
Success	#22C55E
Warning	#F59E0B
Error	#EF4444
Info	#3B82F6

Theme Modes

Mode	Support
Dark Mode	Primary
Light Mode	Optional

Dark mode should be default.

6. Typography System

Primary Font

Font Family

- Inter

Use:

- [Google Fonts — Inter](#)

Typography Scale

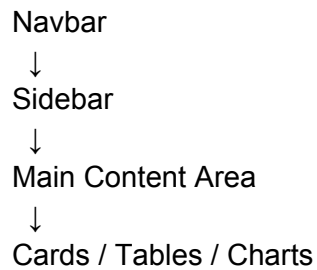
Usage	Size
Hero Title	48px
Section Title	32px
Card Title	20px
Body Text	16px
Small Text	14px

Font Weights

Weight	Usage
700	Headings
600	Buttons
500	Labels
400	Body text

7. Layout System

Layout Structure



Grid System

Screen	Grid
Desktop	12-column
Tablet	8-column
Mobile	4-column

Spacing System

Size	Value
XS	4px
SM	8px
MD	16px
LG	24px

XL 32px

Use consistent spacing everywhere.

8. Component Design System

Buttons

Primary Button

- Blue gradient
- Rounded-lg
- Hover animation
- White text

Secondary Button

- Transparent background
- Border outline
- Hover fill effect

Inputs

Feature	Style
Border Radius	Rounded-lg
Focus State	Blue glow
Padding	Medium
Background	Dark surface

Cards

Property	Style
Radius	16px
Shadow	Soft
Background	Dark elevated surface
Hover Effect	Slight lift

Tables

Feature	Requirement
Sticky Header	Yes
Row Hover	Yes
Pagination	Yes
Search	Yes

Modals

Feature	Requirement
Backdrop Blur	Yes
Animation	Fade + scale

Close Action Top-right

9. Navigation Design

Navbar

Features

- Sticky top navbar
- Logo
- Search bar
- Notification icon
- User profile dropdown

Sidebar

Features

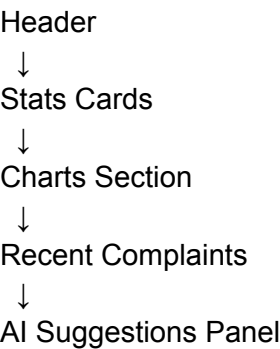
- Collapsible
- Icons + labels
- Active route highlight
- Smooth transitions

Sidebar Menu Items

Dashboard
Complaints
Analytics
Reports
Notifications
Settings
Logout

10. Dashboard UI

Dashboard Layout



Dashboard Components

Component	Description
KPI Cards	Complaint statistics
Charts	Analytics visualization
Tables	Ticket management
Notifications	Updates
AI Suggestions	Smart recommendations

11. Complaint Management UI

Complaint Card Design

Feature	Requirement

Status Badge	Color-coded
Priority Indicator	Visual highlight
Attachment Preview	Thumbnail
Timestamp	Relative time

Complaint Status Design

Status	UI Style
Pending	Yellow badge
In Progress	Blue badge
Resolved	Green badge
Closed	Gray badge

12. Analytics UI

Charts

Use:

- Bar charts
- Pie charts
- Line graphs
- KPI cards

Libraries:

- Chart.js
- Recharts

Analytics Layout

KPI Metrics



Category Distribution



Monthly Trends



Resolution Performance

13. AI Feature UI

AI Suggestion Panel

Feature	Style
AI Badge	Purple gradient
Suggestion Box	Elevated card
Priority Indicator	Colored chip

AI Interaction Style

Should feel:

- Helpful
- Intelligent
- Non-intrusive

14. Responsive Design Requirements

Breakpoints

Device	Width
Mobile	< 640px
Tablet	640px – 1024px
Desktop	> 1024px

Mobile Design Rules

Rule	Requirement
Sidebar	Drawer menu
Tables	Horizontal scroll
Buttons	Full-width where needed
Charts	Responsive resize

15. Accessibility Requirements

Accessibility Standards

Requirement	Description
Keyboard Navigation	Required
Color Contrast	WCAG compliant

ARIA Labels Required

Focus Indicators Visible

Accessibility Features

- High contrast text
- Large click areas
- Screen reader compatibility
- Responsive typography

16. Animations & Transitions

Animation Style

Animation	Usage
Fade In	Page load
Slide Transition	Sidebar
Hover Lift	Cards
Smooth Scaling	Buttons

Animation Rules

- Fast and subtle
- No excessive motion
- Professional feel

Use:

- Framer Motion

Official Website:

- [Framer Motion](#)

17. Empty States & Error States

Empty State Design

Should include:

- Illustration/icon
- Helpful message
- Call-to-action button

Error State Design

Error	UI
404	Friendly illustration
Server Error	Retry button
No Data	Empty chart state

18. Notification System UI

Notification Types

Type	Style
------	-------

Success Green toast

Warning Yellow toast

Error Red toast

Info Blue toast

Notification Position

- Top-right desktop
- Bottom mobile

Use:

- React Hot Toast

Official Website:

- [React Hot Toast](#)

19. Icon System

Recommended Icons

Use:

- Lucide React

Official Website:

- [Lucide Icons](#)

Icon Style

Property	Style
Stroke Width	1.5
Size	18–24px
Theme	Minimal outline

20. Design Inspiration References

Inspiration Platforms

UI Inspiration

- [Dribbble](#)
- [Behance](#)
- [Mobbin](#)

Design References

Inspired by:

- Linear
- Notion
- Discord
- Vercel

21. UI Development Guidelines

Frontend Libraries

Purpose	Library
Components	shadcn/ui
Styling	Tailwind CSS
Icons	Lucide React
Animations	Framer Motion

Recommended UI Stack

React.js
↓
Tailwind CSS
↓
shadcn/ui
↓
Framer Motion

22. Future UI Enhancements

Future improvements:

- Glassmorphism mode
- AI chatbot UI
- Real-time notification center
- Mobile application design
- Theme customization
- Multi-language UI

23. Final UI/UX Summary

SupportSphere AI's UI/UX design is built around:

- Simplicity
- Modern SaaS aesthetics

- AI-first interactions
- Accessibility
- Responsive layouts
- Enterprise-grade workflows

The interface should feel:

- Fast
- Clean
- Smart
- Professional
- Scalable

while maintaining a smooth and intuitive user experience across all devices.

Backend Schema

SupportSphere AI — AI-Powered Smart Complaint & Support Management System

1. Document Overview

Purpose

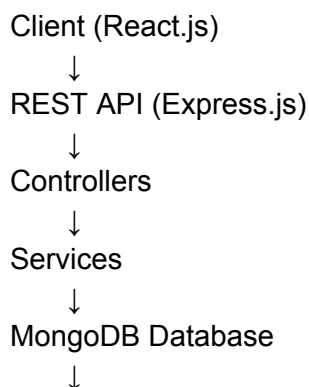
This Backend Schema document defines:

- Database architecture
- Collections & relationships
- Authentication structure
- API data flow
- Role management
- Security rules
- Backend logic structure

This schema is designed for:

- MERN stack development
- Scalable architecture
- AI-assisted workflows
- Cloud deployment

2. Backend Architecture Overview



3. Backend Tech Stack

Layer	Technology
Runtime	Node.js
Framework	Express.js
Database	MongoDB Atlas
ODM	Mongoose
Authentication	JWT
Password Hashing	bcrypt
File Upload	Multer
Cloud Storage	Cloudinary
AI Integration	Gemini API
Validation	express-validator
Security	Helmet + CORS

4. Database Design Overview

Collections

- users
- tickets
- comments

notifications
analytics
auditLogs
attachments

Database Relationships

User
↓
Tickets
↓
Comments
↓
Notifications

5. User Collection Schema

Collection Name

users

Purpose

Stores:

- Authentication data
- Profile information
- Roles & permissions

User Schema Structure

```
{  
  _id: ObjectId,  
  
  name: String,
```

```
email: String,  
  
password: String,  
  
role: String,  
  
avatar: String,  
  
phoneNumber: String,  
  
isVerified: Boolean,  
  
status: String,  
  
lastLogin: Date,  
  
createdAt: Date,  
  
updatedAt: Date  
}
```

User Field Definitions

Field	Type	Description
name	String	Full name
email	String	Unique email
password	String	bcrypt hashed password
role	String	user/admin/support
avatar	String	Profile image
status	String	active/suspended
isVerified	Boolean	Email verification

Role Enum

["user", "admin", "support"]

Status Enum

["active", "inactive", "suspended"]

6. Ticket Collection Schema

Collection Name

tickets

Purpose

Stores all complaints/support requests.

Ticket Schema Structure

```
{  
  _id: ObjectId,  
  
  title: String,  
  
  description: String,  
  
  category: String,  
  
  priority: String,  
  
  status: String,  
  
  attachments: [String],  
}
```

```
aiSummary: String,  
  
aiSuggestion: String,  
  
createdBy: ObjectId,  
  
assignedTo: ObjectId,  
  
resolvedAt: Date,  
  
createdAt: Date,  
  
updatedAt: Date  
}
```

Ticket Field Definitions

Field	Type	Description
title	String	Complaint title
description	String	Full complaint
category	String	Complaint category
priority	String	AI/user assigned priority
status	String	Ticket progress
attachments	Array	File URLs
aiSummary	String	AI-generated summary
aiSuggestion	String	AI-generated solution

Category Enum

```
[
```

```
"technical",  
"hostel",  
"academic",  
"administrative",  
"security",  
"other"  
]
```

Priority Enum

```
["low", "medium", "high", "critical"]
```

Ticket Status Enum

```
[  
"pending",  
"in_progress",  
"resolved",  
"closed"  
]
```

7. Comment Collection Schema

Collection Name

`comments`

Purpose

Stores conversations/replies on complaints.

Comment Schema Structure

```
{
  _id: ObjectId,

  ticketId: ObjectId,

  userId: ObjectId,

  message: String,

  attachments: [String],

  createdAt: Date,

  updatedAt: Date
}
```

Relationships

Field	Relation
ticketId	References tickets
userId	References users

8. Notification Collection Schema

Collection Name

notifications

Purpose

Stores system/user notifications.

Notification Schema

```
{
  _id: ObjectId,

  userId: ObjectId,

  title: String,

  message: String,

  type: String,

  isRead: Boolean,

  createdAt: Date
}
```

Notification Types

```
[
  "ticket_created",
  "ticket_updated",
  "ticket_resolved",
  "new_reply",
  "system_alert"
]
```

9. Analytics Collection Schema

Collection Name

analytics

Purpose

Stores aggregated analytics data.

Analytics Schema

```
{
  _id: ObjectId,

  totalTickets: Number,

  resolvedTickets: Number,

  pendingTickets: Number,

  avgResolutionTime: Number,

  categoryStats: Object,

  monthlyStats: Object,

  generatedAt: Date
}
```

10. Audit Logs Collection Schema

Collection Name

auditLogs

Purpose

Tracks admin/system actions for security and monitoring.

Audit Log Schema

```
{
  _id: ObjectId,
```



```
userId: ObjectId,  
  
action: String,  
  
targetId: ObjectId,  
  
targetType: String,  
  
ipAddress: String,  
  
createdAt: Date  
}
```

Audit Examples

Action	Description
CREATE_TICKET	Ticket creation
UPDATE_STATUS	Status update
DELETE_TICKET	Ticket deletion
LOGIN	User login

11. Attachments Collection Schema

Collection Name

`attachments`

Purpose

Stores uploaded file metadata.

Attachment Schema

```
{
  _id: ObjectId,

  uploadedBy: ObjectId,

  ticketId: ObjectId,

  fileName: String,

  fileUrl: String,

  fileType: String,

  fileSize: Number,

  uploadedAt: Date
}
```

12. Database Relationships

Relationship Diagram

```
User
|
|— creates → Tickets
|— writes → Comments
|— receives → Notifications
|
Ticket
|
|— contains → Comments
|— contains → Attachments
|— assignedTo → Support/Admin
```

13. Authentication Schema

JWT Payload Structure

```
{
  userId,
  role,
  email
}
```

Password Storage

Requirement	Value
Algorithm	bcrypt
Salt Rounds	10

Authentication Flow

```
Login Request
  ↓
Validate User
  ↓
Compare Password
  ↓
Generate JWT
  ↓
Send Token
```

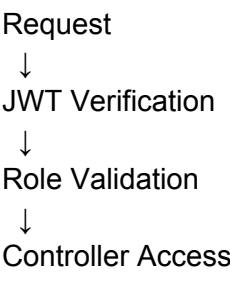
14. Authorization Rules

Access Matrix

Resource	User	Support	Admin
Create Ticket	✓	✓	✓
View Own Tickets	✓	✓	✓

View All Tickets	✕	✓	✓
Update Ticket	✕	✓	✓
Delete Ticket	✕	✕	✓
Analytics Access	✕	✕	✓

Middleware Requirements



15. API Request Validation

Validation Rules

Field	Validation
email	Valid email
password	Min 8 chars
title	Required
description	Required
attachment s	Max 5 files

Validation Middleware

Use:

- `express-validator`

Official Website:

- [express-validator](#)

16. File Upload Architecture

Upload Flow

User Upload



Multer Middleware



Cloudinary Upload



Store URL in MongoDB

File Constraints

Requirement	Limit
Max Size	5MB
Max Files	5
Formats	JPG, PNG, PDF

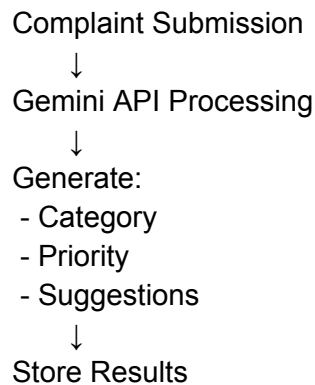
Cloud Storage

Use:

- [Cloudinary](#)

17. AI Integration Schema

AI Processing Pipeline



AI Response Structure

```
{
  category: String,

  priority: String,

  summary: String,

  suggestion: String
}
```

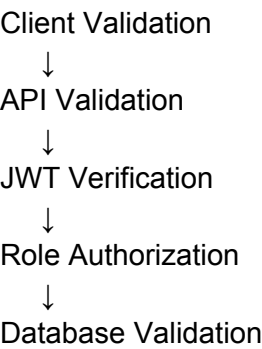
AI Provider

Use:

- [Google Gemini API](#)

18. Security Architecture

Security Layers



Security Middleware

Middleware	Purpose
helmet	Secure headers
cors	Cross-origin control
rate-limit	API abuse protection
dotenv	Environment security

19. Performance Optimization

Optimization Strategies

Strategy	Purpose
Pagination	Large datasets
Indexing	Faster search
Caching	Faster analytics

Lazy Loading Reduced load

Recommended MongoDB Indexes

tickets.status
tickets.category
tickets.createdAt
users.email
notifications.userId

20. Environment Variables Schema

Backend Environment Variables

PORT=5000

MONGO_URI=

JWT_SECRET=

GEMINI_API_KEY=

CLOUDINARY_NAME=

CLOUDINARY_KEY=

CLOUDINARY_SECRET=

NODE_ENV=development

21. Folder Structure

Backend Folder Organization

```
server/  
|  
├── config/  
├── controllers/  
└── middleware/
```



```
|— models/
|— routes/
|— services/
|— validators/
|— uploads/
|— utils/
|— app.js
```

22. API Response Standards

Success Response

```
{
  "success": true,
  "message": "Ticket created successfully",
  "data": {}
}
```

Error Response

```
{
  "success": false,
  "message": "Validation failed",
  "errors": []
}
```

23. Database Scaling Considerations

Future scalability support:

- Sharding
- Redis caching
- Microservices
- WebSockets
- Queue systems
- Event-driven architecture

24. Future Backend Enhancements

Future improvements:

- Real-time notifications
- AI chatbot
- Email services
- SMS alerts
- Multi-tenant architecture
- OCR processing
- Voice complaint handling

25. Final Backend Summary

The SupportSphere AI backend architecture is designed to be:

- Secure
- Scalable
- Modular
- Cloud-ready
- AI-integrated
- Production-friendly

The schema supports:

- Authentication
- Complaint management
- AI-powered automation
- Analytics
- Reporting
- File uploads
- Notifications
- Role-based access control

through a robust MERN-stack backend system architecture.

Implementation Plan

SupportSphere AI — AI-Powered Smart Complaint & Support Management System

1. Document Overview

Purpose

This Implementation Plan defines:

- Development phases
- Build sequence
- Task priorities
- Team workflow
- Testing process
- Deployment strategy
- Timeline & milestones

The goal is to ensure:

- Structured development
- Faster delivery
- Fewer integration issues
- Efficient AI-assisted coding
- Successful production deployment

2. Project Objectives

Primary Goals

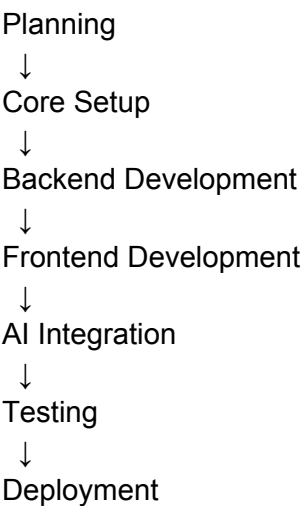
- Build a production-ready MERN stack application
- Complete MVP within 7 days
- Integrate AI-powered automation
- Deploy live application
- Maintain clean architecture

3. Development Methodology

Methodology

Agile + Rapid MVP Development

Development Strategy



4. Project Timeline Overview

Phase	Duration
Planning & Setup	Day 1
Backend Development	Day 2–3
Frontend Development	Day 3–5
AI Integration	Day 5

Testing & Optimization Day 6

Deployment & Final Polish Day 7

5. Development Environment Setup

Required Software

Software	Purpose
Node.js	Runtime environment
Visual Studio Code	Code editor
Git	Version control
MongoDB Atlas	Database
Postman	API testing

Recommended VS Code Extensions

Extension	Purpose
ES7+ React Snippets	React productivity
Tailwind CSS IntelliSense	Tailwind support
Prettier	Formatting
ESLint	Code quality

6. Project Initialization Phase

Goals

- Setup repositories
- Configure environments
- Install dependencies
- Create base architecture

Tasks

Backend Initialization

```
mkdir server  
cd server  
npm init -y
```

Install:

- Express
- Mongoose
- JWT
- bcrypt
- dotenv
- cors
- multer
- cloudinary

Frontend Initialization

```
npm create vite@latest client  
cd client  
npm install
```

Install:

- React Router
- Axios
- Tailwind CSS
- Chart.js
- React Hot Toast

Git Setup

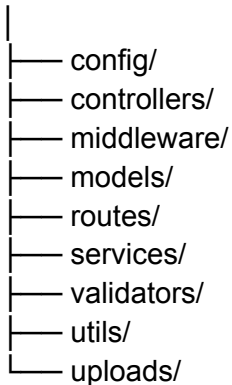
git init

git remote add origin <repo-url>

7. Folder Structure Implementation

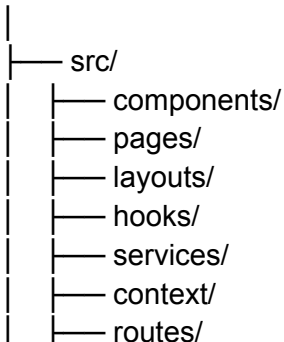
Backend Structure

server/



Frontend Structure

client/



utils/

assets/

8. Phase 1 — Authentication System

Objective

Implement secure user authentication.

Backend Tasks

Task	Status
Create user schema	☐
Register API	☐
Login API	☐
JWT generation	☐
Password hashing	☐
Role middleware	☐

Frontend Tasks

Task	Status
Login page	☐

Register page ☐

Form validation ☐

Token storage ☐

Protected routes ☐

Authentication Flow

Register/Login
↓
Validate Inputs
↓
Generate JWT
↓
Store Token
↓
Redirect Dashboard

9. Phase 2 — Complaint Management System

Objective

Build ticket creation and tracking system.

Backend Tasks

Task	Status
Ticket schema	☐

CRUD APIs ☐

Validation middleware ☐

File upload support ☐

Frontend Tasks

Task	Status
Complaint form	☐
Complaint list	☐
Complaint details page	☐
Status tracking UI	☐

Complaint Flow

Create Complaint
↓
Submit Form
↓
Store in MongoDB
↓
Display in Dashboard

10. Phase 3 — Admin Dashboard

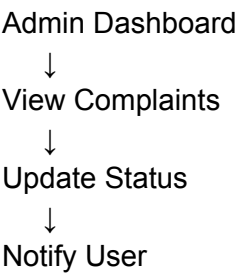
Objective

Allow admins to manage complaints.

Features

Feature	Status
View all complaints	☐
Search/filter	☐
Update statuses	☐
Assign priorities	☐
Reply to users	☐

Admin Workflow



11. Phase 4 — AI Integration

Objective

Add intelligent automation using AI.

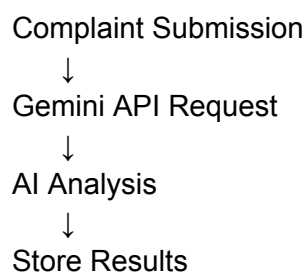
AI Features

Feature	Purpose
Categorization	Detect issue type
Priority Prediction	Estimate urgency
AI Suggestions	Smart solutions
Summarization	Short summaries

AI Integration Tasks

Task	Status
Gemini API setup	☐
AI service layer	☐
Prompt engineering	☐
Store AI responses	☐

AI Processing Flow



AI Provider

Use:

- [Google Gemini API](#)

12. Phase 5 — Analytics Dashboard

Objective

Create analytics & reporting system.

Analytics Features

Feature	Status
KPI cards	☐
Pie charts	☐
Line graphs	☐
Monthly reports	☐

Analytics Metrics

Metric	Purpose
Total complaints	Platform activity

Pending tickets	Workload
Resolution time	Efficiency
Category distribution	Issue trends

Chart Libraries

Use:

- Chart.js
- Recharts

13. Phase 6 — File Upload System

Objective

Enable attachment uploads.

Tasks

Task	Status
Multer setup	☐
Cloudinary integration	☐
File validation	☐
Upload preview UI	☐

Upload Flow

User Upload
↓
Multer Middleware
↓
Cloudinary Upload
↓
Save URL

Storage Platform

Use:

- [Cloudinary](#)

14. Phase 7 — PDF Report Generation

Objective

Generate downloadable reports.

Report Types

Report	Purpose
Complaint Summary	Ticket details
Analytics Report	Statistics
User History	Complaint records

PDF Tasks

Task	Status
PDF library setup	☐
Report templates	☐
Export buttons	☐

PDF Libraries

Library	Usage
jsPDF	Frontend reports
PDFKit	Backend reports

15. Phase 8 — UI/UX Enhancement

Objective

Polish user interface.

Tasks

Task	Status
Responsive layouts	☐

Animations	☐
Dark mode	☐
Empty states	☐
Error states	☐

UI Libraries

Library	Purpose
Tailwind CSS	Styling
Framer Motion	Animations
Lucide React	Icons

16. Phase 9 — Security Implementation

Objective

Secure application infrastructure.

Security Tasks

Task	Status
JWT protection	☐

Password hashing ☐

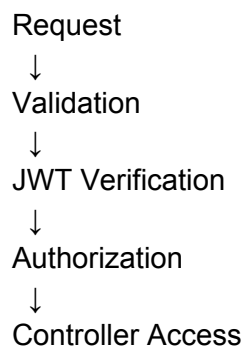
Rate limiting ☐

Input validation ☐

CORS setup ☐

Helmet security ☐

Security Flow



17. Phase 10 — Testing & Debugging

Objective

Ensure application stability.

Testing Types

Type	Tool
API Testing	Postman

UI TestingReact Testing Library

Manual TestingBrowser

Authentication TestingJWT validation

Testing Checklist

Test	Status
Login works	☐
CRUD works	☐
File uploads work	☐
AI responses work	☐
Analytics load	☐
Mobile responsive	☐

18. Phase 11 — Deployment

Objective

Deploy production-ready application.

Deployment Targets

Layer	Platform
Frontend	Vercel
Backend	Render
Databases	MongoDB Atlas

Deployment Flow

GitHub Repository



Frontend → Vercel

Backend → Render

Database → MongoDB Atlas

Deployment Tasks

Task	Status
Configure environment variables	☐
Setup production build	☐
Test APIs	☐
Configure domains	☐

19. Environment Variables Setup

Backend .env

PORT=5000

MONGO_URI=

JWT_SECRET=

GEMINI_API_KEY=

CLOUDINARY_NAME=

CLOUDINARY_KEY=

CLOUDINARY_SECRET=

Frontend .env

VITE_API_URL=

VITE_GEMINI_API_KEY=

20. GitHub Workflow

Git Branch Structure

main

```
|— dev
|— feature/auth
|— feature/tickets
|— feature/analytics
|— feature/ai
```

Commit Naming Convention

Type	Example
Feature	feat: add auth API
Fix	fix: resolve login bug
UI	style: improve dashboard

21. Daily Development Plan (7-Day MVP)

Day 1

Setup + Authentication

- Project initialization
- MongoDB connection
- JWT authentication
- Login/Register UI

Day 2

Complaint System

- Ticket schema
- CRUD APIs
- Complaint pages

Day 3

Admin Dashboard

- Complaint management
- Status updates
- Filtering/search

Day 4

AI Integration

- Gemini API
- AI suggestions
- Categorization

Day 5

Analytics + Charts

- Dashboard KPIs
- Graphs
- Reports

Day 6

File Upload + PDFs

- Cloudinary integration
- PDF generation
- Export system

Day 7

Testing + Deployment

- Bug fixes
- Responsive testing
- Deploy frontend/backend
- Final polish

22. MVP Completion Checklist

Feature	Status
---------	--------

Authentication	☐
Ticket CRUD	☐
Admin Dashboard	☐
AI Features	☐
Analytics	☐
File Uploads	☐
PDF Reports	☐
Deployment	☐

23. Future Enhancements

Future development roadmap:

- Real-time notifications
- AI chatbot
- Mobile application
- Email alerts
- OCR image processing
- Voice complaints
- Multi-language support

24. Risk Management

Risk	Solution
API failure	Retry handling

AI inaccuracies	Manual review
Upload abuse	File limits
Performance issues	Pagination & indexing

25. Final Implementation Summary

The SupportSphere AI implementation plan is designed for:

- Rapid MVP delivery
- Scalable architecture
- AI-assisted development
- Production deployment
- Clean workflows

The phased implementation ensures:

- Stable development
- Modular architecture
- Easy debugging
- Faster testing
- Efficient deployment

while maintaining professional full-stack development standards.